2025/05/13/AI_R&D_Center

ナップサック問題における一次元DPによるメモリ削減

23B1014　旦保惟果

やりたいことを、できることに。
湘南工科大学
SHONAN INSTITUTE OF TECHNOLOGY
AI R&D CENTER

目次

- 動的計画法とは
- 0-1ナップサック問題
- 二次元DPを用いた基本解法
- 二次元DPの問題点
- 改良された二次元DP
- 一次元DPでの解法
- なぜ j を逆順に回す必要があるのか

動的計画法とは

- **動的計画法**(Dynamic Programming, 以下DP)は, 大きな問題を小さな部分問題に分割して解く手法

- 一度計算した答えを保存しておくことで, 何度も同じ計算をしなくて済む
  ⇒計算量が少なくなる

- 例
  - グラフの最短経路問題
  - 日本語形態素解析
  - 部分和問題, ナップサック問題
  - フィボナッチ数列

0-1ナップサック問題

- 価値が v_i 重さが w_i の N 個の荷物と, 容量 W のナップサックがある.
- 次の条件を満たすように品物を選んでナップサックに入れたい.
  - 選んだ品物の価値の合計をできるだけ大きくする.
  - 選んだ品物の重さの総和は W を超えてはならない.
  - 同じ品物を複数回選択してはいけない.

- 価値の総和の最大値を求めてください.

二次元DPを用いた基本解法

- 今回のナップサック問題では2つの情報を保持したい
  - どこまでの品物を見たか(i 個目まで)
  - 今のナップサックの重さはどのくらいか(j kg)

- 品物とナップサックの重さの2つを小さいものから順番に決定して答えを求める

- $dp[i][j] \coloneqq i$ 番目までの品物を考えて, 重さが j になるときの価値の総和の最大値

- この式をもとにして品物iを選ぶ・選ばない場合の2つを考える

二次元DPを用いた基本解法

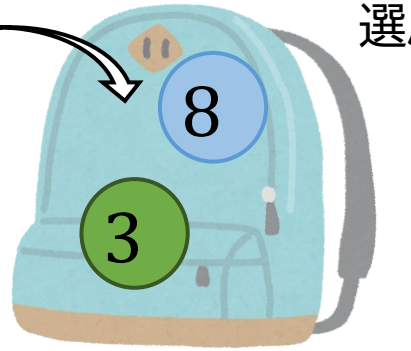
- 選ぶ場合 $(w_i + j \leq W)$

  - $dp[i][j + w_i] = \max \begin{cases} dp[i-1][j + w_i] & \text{（品物 } i \text{ を選ばないで重さ } j + w_i \text{ ）} \\ dp[i-1][j] + v_i & \text{（品物 } i \text{ を選んで重さ } j + w_i \text{ ）} \end{cases}$

二次元DPを用いた基本解法

重さ:$j + w_i$
価値:$i - 1$番目までの品物を適切に
選んで価値最大

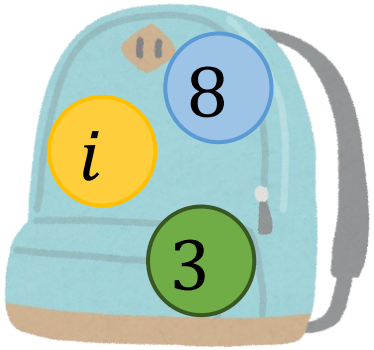
重さ:j
価値:$i - 1$番目までの品物を適切に
選んで価値最大



i

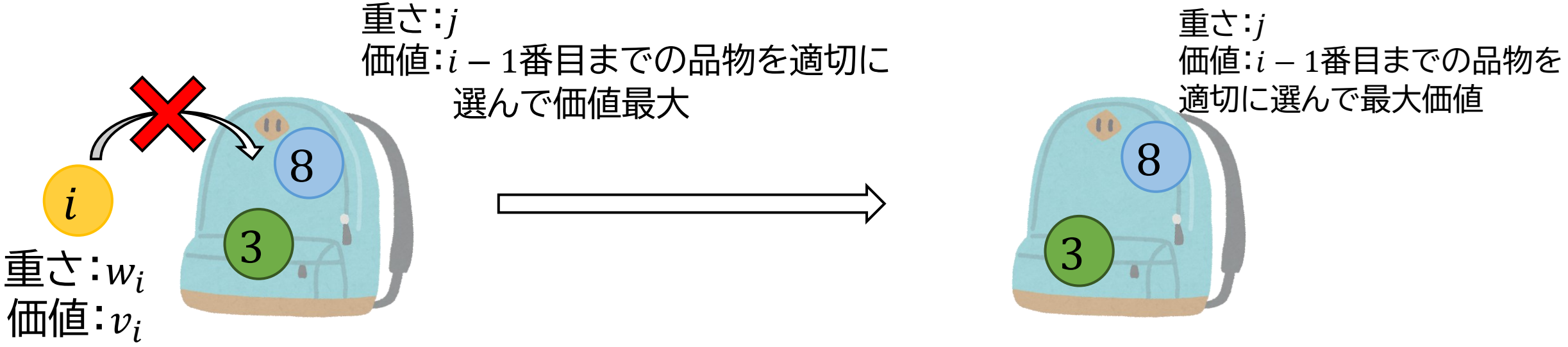
重さ:w_i
価値:v_i

大きい方をとる



二次元DPを用いた基本解法

- 選ばない場合
  - $dp[i][j] = dp[i-1][j]$ (品物 i を選ばない場合)



二次元DPを用いた基本解法

- 漸化式

$$dp[i][j] = dp[i-1][j]\ (j < w_i)$$

$$dp[i][j+w_i] = \begin{cases} max\{dp[i-1][j] + v_i, dp[i-1][j+w_i]\}\ (j + w_i \leq W) \\ dp[i-1][j+w_i]\ (j + w_i > W) \end{cases}$$

二次元DPの問題点

	0	j	…	$j+w_i$	…	W
0	$dp[0][0]$	…	…	…	…	$dp[0][W]$
⋮	⋮	⋮	⋮	⋮	⋱	⋮
$i-1$	$dp[i-1][0]$	$dp[i-1][j]$	…	$dp[i-1][j+w_i]$	…	$dp[i-1][W]$
i	$dp[i][0]$	…	…	$dp[i][j+w_i]$	…	$dp[i][W]$

- $0 \sim i$ 行目の情報を保存するために, メモリを $O(NW)$ 使用している

- i行目の更新に必要なのは $i-1$ 行目だけなので, $i-1$ 行目と i 行目の情報のみを管理する
  ⇒空間計算量を削減できる

改良された二次元DP

	0	j	…	$j+w_i$	…	W
$i-1$	$dp[i-1][0]$	$dp[i-1][j]$	…	$dp[i-1][j+w_i]$	…	$dp[i-1][W]$
i	$dp[i][0]$	…	…	$dp[i][j+w_i]$	…	$dp[i][W]$

↓

	0	j	…	$j+w_i$	…	W
i	$dp[i][0]$	$dp[i][j]$	…	$dp[i][j+w_i]$	…	$dp[i][W]$
$i+1$	$dp[i+1][0]$	…	…	$dp[i+1][j+w_i]$	…	$dp[i+1][W]$

- 情報を保持しておくのは，現在の行と一つ前の行だけでもうまくいく

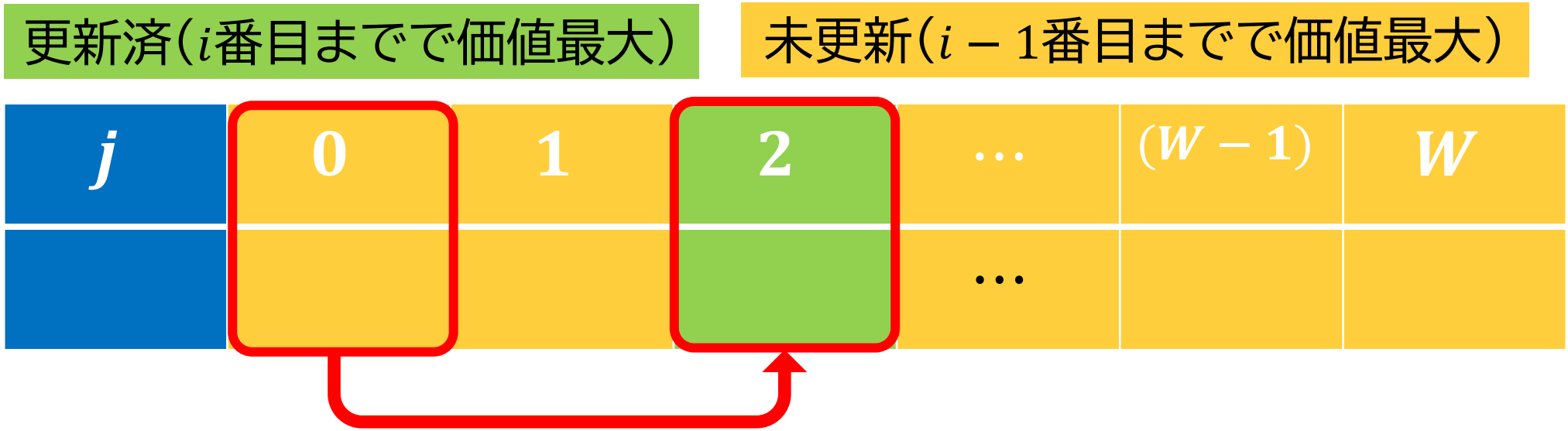
一次元DPでの解法

	0	j	…	$j+w_i$	…	W
	$dp[i-1][0]$	$dp[i-1][j]$	…	$dp[i-1][j+w_i]$	…	$dp[i-1][W]$

- さらに, 更新の順番を考えることで保持する情報が一行で済む
- 一行の中に i 行目と $i-1$ 行目の情報を両方入れ, 更新した情報を上書きしていく
- j を逆順 $(W, W-1, \ldots, w_i)$ に回すことで, 正しい情報の更新が可能になる

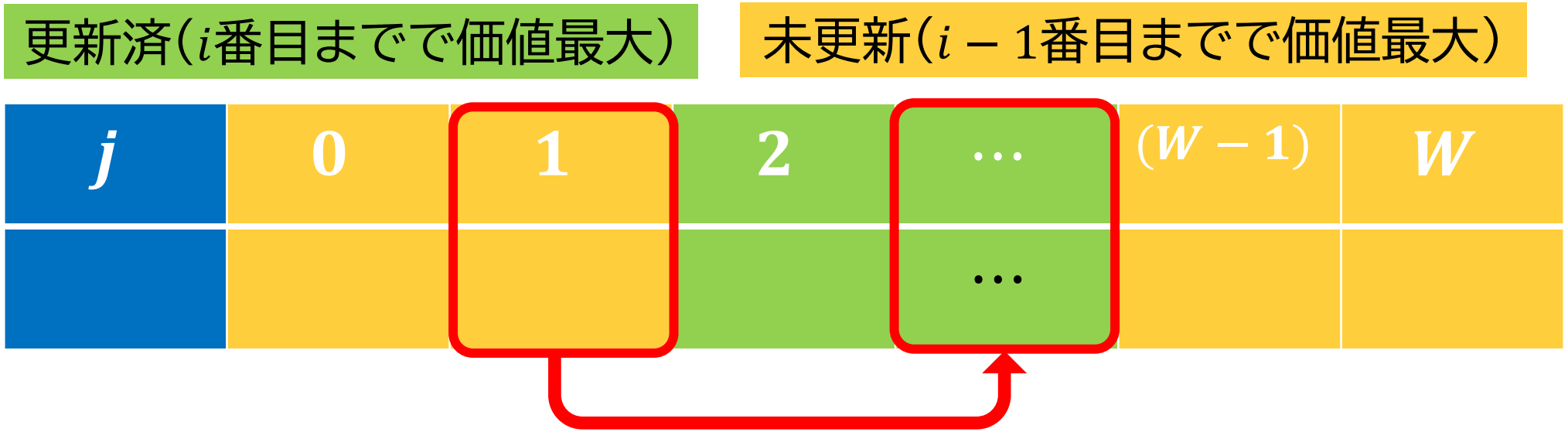
なぜjを逆順に回す必要があるのか

- 順に更新していく



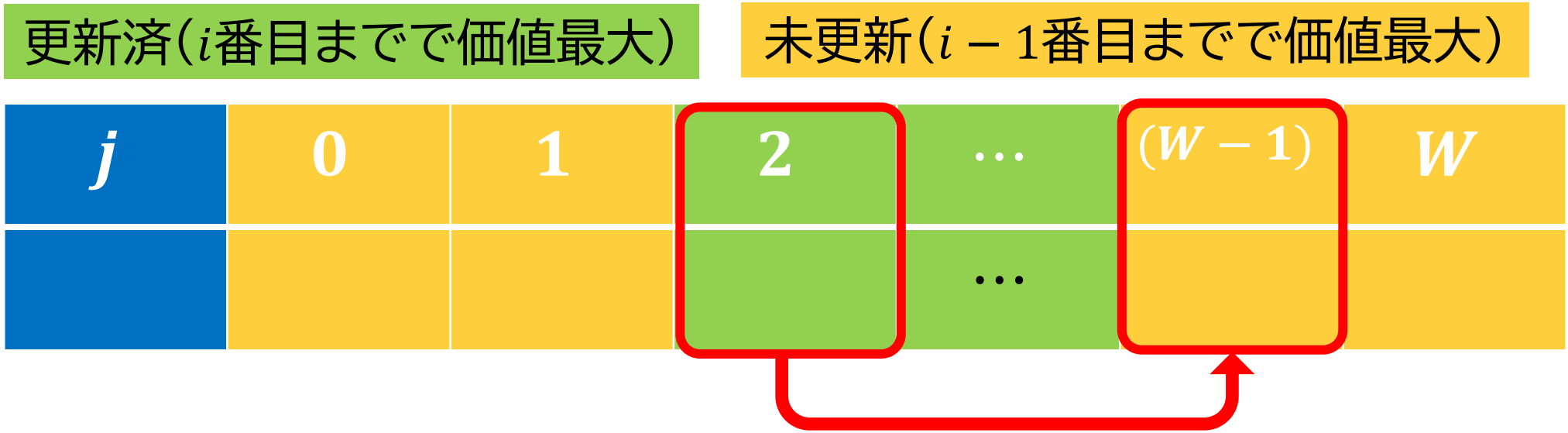
なぜjを逆順に回す必要があるのか

- 順に更新していく



なぜjを逆順に回す必要があるのか

- 順に更新していく



- i番目の品物を入れた状態のナップサックにi番目の品物を入れていることになる
  ⇒今回の条件を満たしていない(同じ品物を複数個選んではいけない)

なぜjを逆順に回す必要があるのか

- 逆順に更新していく

未更新（$i-1$番目までで価値最大）　更新済（i番目までで価値最大）

j	0	1	2	…	$(W-1)$	W
				…		

なぜjを逆順に回す必要があるのか

- 逆順に更新していく

未更新($i-1$番目までで価値最大)　更新済(i番目までで価値最大)

j	0	1	2	…	$(W-1)$	W
				…		

なぜjを逆順に回す必要があるのか

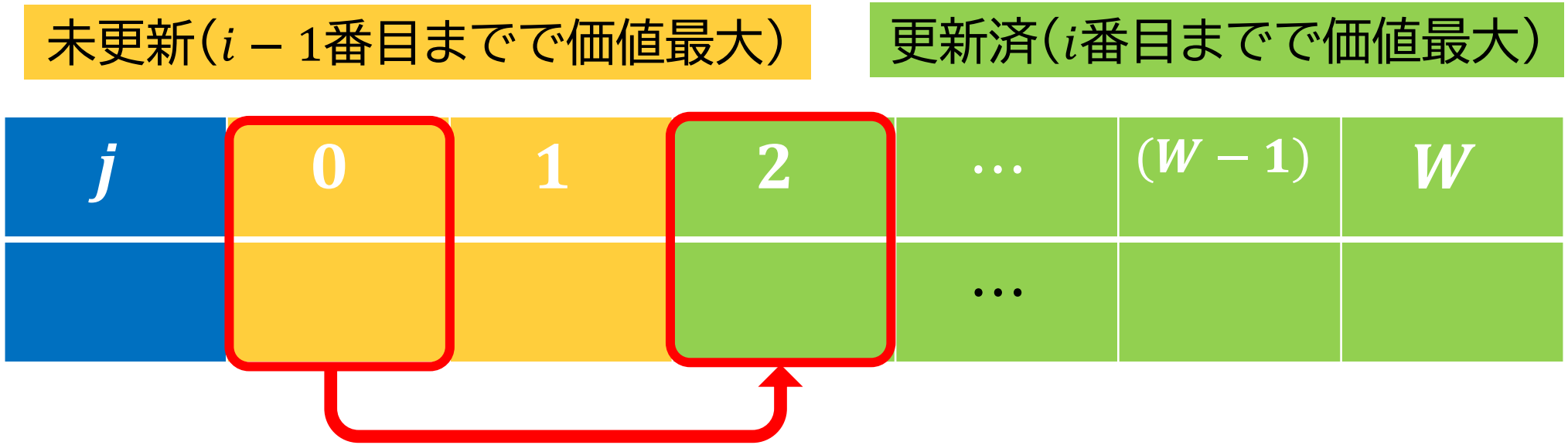
- 逆順に更新していく

未更新（$i-1$番目までで価値最大）　更新済（i番目までで価値最大）

j	0	1	2	…	$(W-1)$	W
				…		

なぜjを逆順に回す必要があるのか

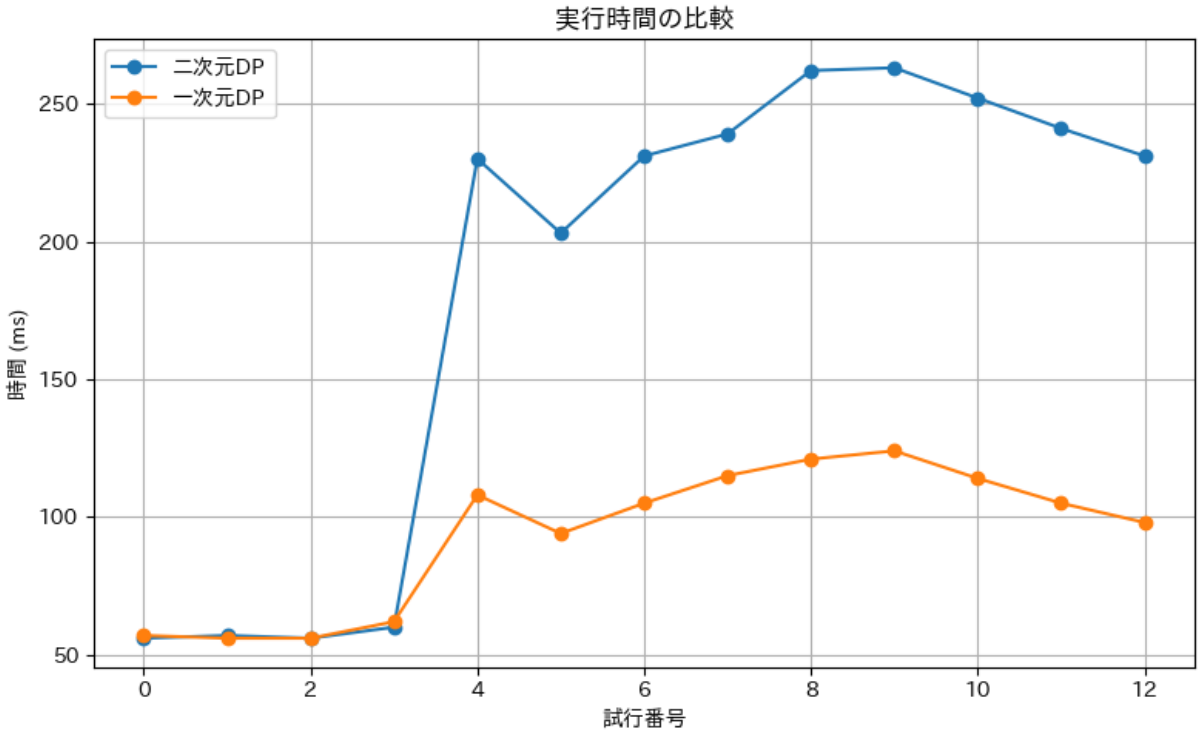
- 逆順に更新していく



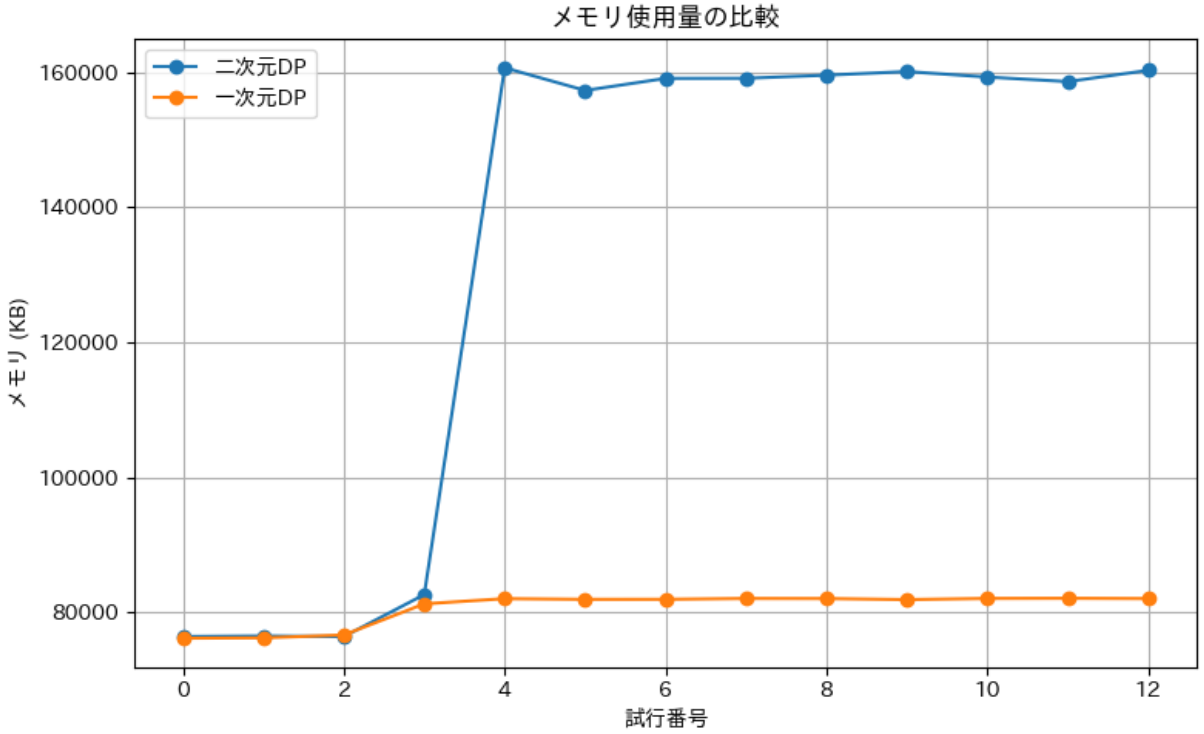
- 「$i-1$番目の品物で重さjになるもの」から「i番目の品物で重さjになるもの」の遷移で考えている
  →二次元DPのときの同じ遷移ができている

実行結果

実行時間の比較



メモリの比較



一次元DPのほうが明らかに早いことがわかる

まとめ

- 二次元⇒一次元にすることで, 空間計算量が $o(W)$ に削減できる
  - 大規模なサイズのデータにも対応できる
  - 限られたメモリ環境の中でも実装が可能
  - キャッシュ効率が良くなり, メモリアクセス量も減ることで実行時間の向上につながる